

Budget-Funnelled Monte-Carlo Tree Search

A wide-to-narrow cascade that spends a fixed simulation budget where it matters

Louay Alsakka · 2026 · *working paper / technical report*

Abstract

Monte-Carlo Tree Search (MCTS) with a policy+value network spends most of its simulation budget re-searching moves that a shallow look already reveals to be poor. We describe a simple **staged cascade**: instead of one flat search over all legal moves, a fixed budget of B simulations is run in N stages that funnel from **wide-and-shallow to narrow-and-deep** — early stages look at many candidate moves briefly and prune the obvious junk; later stages spend the remaining budget only on the handful of survivors, searched deeper and more greedily. Because expensive deep search (and the value-net evaluations it drives) is confined to a shrinking candidate set, the same nominal budget resolves in far less wall-clock per move. On a small in-house convolutional policy+value network evaluated against a Stockfish rating ladder, a 10-stage cascade reaches the **same playing strength as flat MCTS within measurement noise while using $\sim 4.8\times$ less time per move** (275 ms vs. 1330 ms at a fixed 800-simulation budget). We report this as a *promising, not yet statistically established* result: our per-configuration Elo estimates carry ± 89 -point uncertainty, so the honest claim is *no measurable strength loss at large compute savings*. We outline the experiments needed to make it rigorous — fixed-budget head-to-head matches at scale, and replication inside a strong open-source engine (KataGo, Leela Zero/Leela Chess Zero).

1. Introduction

Test-time search is the dominant lever in modern game-playing systems: given a fixed network, more simulations buy more strength [Silver 2018; Jones 2021]. But not all simulations are equally useful. In a typical position most legal moves are quickly seen to be bad; a flat MCTS nonetheless allocates exploration to them through the PUCT prior [Rosin 2011; Silver 2017], and — more importantly for wall-clock — expands and evaluates their subtrees with the value network. The expensive resource is the **network evaluation per newly expanded leaf**, and a flat search spreads it thinly across the full move set.

This paper studies a minimal way to concentrate that resource: a **budget-funnelled cascade**. The idea is old in spirit — humans and classical engines both narrow a candidate list

before calculating deeply — but we phrase it as an explicit *schedule over search shape* on top of an AlphaZero-style policy+value MCTS, with the number of stages as a single free knob. Our contribution is (i) the concrete formulation, (ii) an efficiency measurement on a chess network against a calibrated Stockfish ladder, and (iii) an honest account of what it does and does not yet establish.

2. Related work

MCTS and selectivity. MCTS originates with Coulom's selectivity-and-backup formulation [Coulom 2006] and the UCT bandit analysis of Kocsis and Szepesvári [2006]; see Browne et al. [2012] for a survey. Our cascade is a search-*control* strategy layered on standard PUCT MCTS [Rosin 2011] as used in AlphaGo/AlphaZero [Silver 2016; Silver 2017; Silver 2018].

Narrowing the candidate set. Progressive strategies — progressive widening and progressive unpruning — grow the considered move set as visits accumulate [Coulom 2007; Chaslot 2008]; our cascade runs the *opposite* schedule at the root, starting wide and *shrinking* the candidate set across stages, while re-investing the freed budget in depth. Rapid Action Value Estimation shares information across moves to focus search [Gelly 2011]; playout-cap and simulation-count variation are used in KataGo for training efficiency [Wu 2019]. Classical alpha-beta engines achieve related focus via iterative deepening and candidate pre-selection; the novelty here is packaging wide→narrow funnelling as a tunable multi-stage schedule for neural MCTS.

Test-time compute. That search scales strength roughly log-linearly in simulations is documented for board games [Jones 2021] and echoed by recent inference-time-compute work in language models. The cascade targets the *efficiency* of that curve — the same strength at lower cost — rather than its ceiling.

3. Method

Let a position have legal moves M , a policy+value network f , and a total budget of B simulations. A cascade is a list of N stages, each a triple (k_i, s_i, c_i) :

- k_i — the number of candidate root moves the stage is allowed to search (the *width*);
- s_i — simulations spent in the stage, with $\sum s_i = B$;
- c_i — the PUCT exploration constant for the stage.

The candidate set starts as the top- k_1 moves by policy prior. Stage i runs s_i PUCT simulations restricted to its candidate set; the top- k_{i+1} moves by visit count then pass to the next stage. Visit counts and the value cache carry across stages, so deeper stages refine — rather than restart — the estimates on survivors. After the final stage ($k_N = 1$) the most-visited move is played.

We use one monotone rule so that N is the only knob: widths shrink geometrically $20 \rightarrow 1$, exploration falls linearly $c: 4.5 \rightarrow 0.2$ (broad early, greedy late), and the budget is weighted toward the later, narrower stages ($\sum s_i = B = 800$). $N = 1$ recovers plain flat MCTS-800. Intuitively, the wide-shallow front cheaply discards junk, and the deep-narrow tail spends the value-net evaluations only on moves that survived scrutiny.

4. Experimental setup

- **Network.** A 3.45M-parameter convolutional policy+value network (raw single-pass strength ≈ 2262 Elo on the ladder below), deliberately small; MLX on Apple Silicon.
- **Opponents / rating.** Matches against a Stockfish `UCI_Elo` ladder (rungs 2400/2700/3000) plus a random-mover anchor; model Elo is a maximum-likelihood performance rating over the ladder. **20 games per rung**, 40 ms/move for the Stockfish side, real Lichess openings.
- **Budget.** A fixed $B = 800$ simulations for every configuration, so the comparison isolates *how* the budget is spent, not *how much*.
- **Speed.** Wall-clock ms/move measured on a fixed probe set (opening, middlegame, tactical).

5. Results

N	Elo (± 89)	ms/move	speedup	schedule (width $\times$ sims, wide $\rightarrow$ narrow)
1 (flat)	2683	1330	1.0 $\times$	800 sims over all moves
3	2605	903	1.5 $\times$	20 $\rightarrow$ 4 $\rightarrow$ 1
4	2683	724	1.8 $\times$	20 $\rightarrow$ 7 $\rightarrow$ 3 $\rightarrow$ 1
7	2605	475	2.8 $\times$	20 $\rightarrow$... $\rightarrow$ 1 (7 stages)
9	2543	300	4.4 $\times$	20 $\rightarrow$... $\rightarrow$ 1 (9 stages)
10	2570	275	4.8$\times$	20 $\rightarrow$... $\rightarrow$ 1 (10 stages)

Two readings, kept separate on purpose:

- **Solid (measured):** at a fixed 800-simulation budget, funnelling the budget wide $\rightarrow$ narrow cuts **wall-clock per move by up to 4.8 $\times$** (1330 $\rightarrow$ 275 ms). This is a direct, low-variance measurement — the narrowed search performs far fewer new value-net expansions for the same nominal budget.
- **Suggestive (not yet significant):** across the sweep, Elo stays in a band (2506–2683) whose members are **statistically indistinguishable** — every estimate carries ± 89 (20 games/rung), so the honest statement is *no measurable strength loss* out to $\sim 4.8\times$

speedup, not a proven equality. Points such as $N=4$ (2683, 1.8 $\times$) and $N=7$ (2605, 2.8 $\times$) match flat MCTS most cleanly.

6. Limitations and the path to a rigorous result

We deliberately under-claim. Three gaps must be closed before this is a defensible efficiency result:

1. **Statistical significance.** ± 89 Elo from 20 games/rung is too coarse. The clean test is a **direct head-to-head match, cascade vs. flat MCTS at an identical budget**, over thousands of games, reporting an Elo difference with confidence intervals (and, for the efficiency claim, matched *strength* at measured *nodes/wall-clock*).
2. **One small network, one game.** Results on a 3.45M chess net may not transfer. The convincing demonstration is replication inside a **state-of-the-art open-source engine — KataGo or Leela Zero/Leela Chess Zero** — showing an improvement (or a compute saving at equal strength) against that engine's own flat MCTS.
3. **Confounds.** Wall-clock mixes cache effects and batch sizes; a fair comparison should also report value-net evaluations and node counts, and control the batched-evaluation path identically across arms.

We view the present numbers as motivation to run (1)–(2), not as a substitute for them.

7. Conclusion

Spending a fixed MCTS budget in a wide-to-narrow cascade concentrates the costly value-net evaluations on moves that survive a cheap wide pass, and on a small chess network it preserves playing strength (within measurement noise) at up to 4.8 $\times$ less time per move. The idea is simple, orthogonal to the network, and has one interpretable knob. Whether it yields a *statistically significant* gain on a strong reference engine is the open question we intend to answer next.

Acknowledgements

We thank Rémi Coulom for feedback that sharpened the scope and the evidentiary bar of this note.

References

- Browne, C. et al. (2012). *A Survey of Monte Carlo Tree Search Methods*. IEEE Trans. Comput. Intell. AI Games 4(1).

- Chaslot, G. et al. (2008). *Progressive Strategies for Monte-Carlo Tree Search*. New Mathematics and Natural Computation 4(3).
- Coulom, R. (2006). *Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search*. Computers and Games 2006.
- Coulom, R. (2007). *Computing Elo Ratings of Move Patterns in the Game of Go*. ICGA Journal 30(4).
- Gelly, S., Silver, D. (2011). *Monte-Carlo Tree Search and Rapid Action Value Estimation in Computer Go*. Artificial Intelligence 175(11).
- Jones, A. L. (2021). *Scaling Scaling Laws with Board Games*. arXiv:2104.03113.
- Kocsis, L., Szepesvári, C. (2006). *Bandit Based Monte-Carlo Planning*. ECML 2006.
- Rosin, C. D. (2011). *Multi-Armed Bandits with Episode Context*. Annals of Mathematics and Artificial Intelligence 61(3).
- Silver, D. et al. (2016). *Mastering the Game of Go with Deep Neural Networks and Tree Search*. Nature 529.
- Silver, D. et al. (2017). *Mastering the Game of Go without Human Knowledge*. Nature 550.
- Silver, D. et al. (2018). *A General Reinforcement Learning Algorithm that Masters Chess, Shogi and Go through Self-Play*. Science 362.
- Wu, D. J. (2019). *Accelerating Self-Play Learning in Go*. arXiv:1902.10565 (KataGo).

Software resources. Stockfish (<https://stockfishchess.org>) · Leela Chess Zero (<https://lczero.org>) · KataGo (<https://github.com/lightvector/KataGo>) · MLX (<https://github.com/ml-explore/mlx>). Code and the cascade sweep for this note: <https://github.com/LouayAlsakka/efficient-thinking>.